

Analiză experimentală a performanței unor algoritmi de sortare

Florian Budurean
`florian.budurean06@e-uvt.ro`

Mai 2026

Universitatea de Vest din Timișoara
Facultatea de Informatică

Cuprins

1	Introducere	3
2	Prezentarea algoritmilor	4
2.1	Bubble Sort	4
2.2	Insertion Sort	4
2.3	Merge Sort	4
2.4	Quick Sort	4
2.5	Radix Sort	4
3	Implementare	5
3.1	Reprezentarea datelor	5
3.2	Generarea datelor de test	5
3.3	Măsurarea timpului	5
3.4	Precizări	6
4	Studiu de caz	6
4.1	Condiții de rulare	6
4.2	Rezultate	6
4.2.1	Liste cu elemente aleatoare	6
4.2.2	Liste sortate crescător	7
4.2.3	Liste plate	8
4.3	Analiza rezultatelor și concluzii	9
4.3.1	Liste aleatoare	9
4.3.2	Liste sortate crescător	9
4.3.3	Liste Plate	10
4.4	Observații	10
4.5	Limitările experimentului	10
5	Comparație cu literatura	10
5.1	Problemă	11
5.2	Context	11
5.3	Metodă	11
6	Concluzii	11

Rezumat

Lucrarea prezintă o analiză experimentală a performanței unor algoritmi clasici de sortare: Bubble Sort, Insertion Sort, Merge Sort, Quick Sort și Radix Sort. Algoritmii au fost implementați în C++, iar timpii de execuție au fost măsurați folosind biblioteca `chrono`. Testele au fost realizate pe mai multe tipuri de date de intrare: liste aleatoare, liste sortate crescător și liste plate. Rezultatele confirmă diferențele teoretice dintre algoritmii cu complexitate pătratică și algoritmii mai eficienți, de tip $O(n \log n)$ sau aproape liniar. De asemenea, experimentul evidențiază influența distribuției datelor și a detaliilor de implementare asupra performanței reale, în special în cazul algoritmului Quick Sort.

Cuvinte cheie: algoritmi de sortare, analiză experimentală, Bubble Sort, Insertion Sort, Merge Sort, Quick Sort, Radix Sort, complexitate temporală, C++, benchmark.

1 Introducere

Sortarea este una dintre problemele de bază ale informaticii și apare în foarte multe situații practice, de la baze de date și sisteme de căutare până la procesarea fișierelor sau chiar analiza datelor. De multe ori, performanța unui program depinde și de modul în care sunt organizate datele, iar alegerea unui algoritm de sortare potrivit poate avea un impact important, mai ales atunci când se lucrează cu volume mari de date.

Problema sortării constă în reordonarea unei colecții de elemente după un anumit criteriu, de obicei în ordine crescătoare sau descrescătoare. Deși la prima vedere problema pare simplă, există mai multe moduri de rezolvare, iar fiecare algoritm se comportă diferit în funcție de dimensiunea datelor, de ordinea inițială a valorilor și de felul în care este implementat.

În literatura de specialitate există deja multe soluții cunoscute pentru această problemă. Algoritmi precum Bubble Sort și Insertion Sort sunt ușor de înțeles și de implementat, dar devin ineficienți atunci când numărul de elemente crește. Merge Sort și Quick Sort sunt algoritmi mai performanți, având în general complexitate $O(n \log n)$, iar Radix Sort poate avea o performanță aproape liniară pentru date numerice cu un număr limitat de cifre.

Chiar dacă aceste complexități sunt cunoscute teoretic, în practică rezultatele pot să difere în funcție de mai mulți factori, precum tipul datelor de intrare, alegerea pivotului în Quick Sort, costurile de alocare a memoriei, limbajul de programare sau calculatorul pe care se rulează testele. Din acest motiv, este utilă dezvoltarea unei soluții proprii, ce constă într-un mediu în care algoritmi să fie testați în aceleași condiții și comparați direct.

Soluția propusă în această lucrare este o aplicație realizată în C++, care permite generarea unor seturi de date de test, rularea mai multor algoritmi de sortare și măsurarea timpilor de execuție cu ajutorul bibliotecii `chrono`. Pentru dezvoltarea soluției, au fost implementați algoritmii, au fost generate date de intrare de dimensiuni diferite, iar apoi au fost rulate teste pentru a observa diferențele de performanță dintre aceștia.

Contribuțiile proprii ale lucrării sunt:

- implementarea într-un cadru comun a cinci algoritmi clasici de sortare;
- generarea mai multor tipuri de date de intrare: aleatoare, sortate crescător și plate;
- realizarea unui mecanism de benchmark pentru calcularea timpilor medii de execuție;
- compararea comportamentului algoritmilor în funcție de dimensiunea și structura datelor;
- evidențierea diferențelor dintre comportamentul teoretic și rezultatele practice obținute.

Lucrarea este organizată astfel: mai întâi sunt prezentați algoritmii de sortare analizați și complexitățile acestora. Apoi este descrisă implementarea aplicației, modul de generare a datelor și metoda folosită pentru măsurarea timpului. În continuare sunt prezentate rezultatele experimentale și interpretarea acestora. Spre final, rezultatele obținute sunt comparate cu literatura de specialitate, iar ultima secțiune prezintă concluziile, dificultățile întâlnite și posibile direcții de dezvoltare viitoare.

2 Prezentarea algoritmilor

2.1 Bubble Sort

În timp ce nu are un autor unic, Bubble Sort este mai degrabă un algoritm apărut în literatura informatică timpurie. Este menționat și popularizat în lucrări precum cea a lui Knuth [1]

Algoritmul de sortare „Bubble Sort” este cel mai ușor de înțeles. Acesta parcurge repetat un șir de valori date, comparând și interschimbând elemente vecine care nu sunt aflate în ordinea dorită (crescător sau descrescător, după caz). Procesul se repetă până când întreg șirul de valori este parcurs, iar toate valorile se află ordonate.

Complexitate:

$O(n^2)$ – cazul mediu și cazul defavorabil (elementele din listă sunt sortate invers față de rezultatul dorit).

$O(n)$ – cazul favorabil (lista este deja sortată, iar algoritmul doar parcurge șirul o dată pentru a verifica sortarea).

2.2 Insertion Sort

Insertion Sort este de asemenea un algoritm clasic, folosit instinctiv de către oameni. Acesta este analizat în lucrarea lui Knuth. [1]

Algoritmul de sortare prin inserție construiește progresiv un subșir sortat prin inserarea elementului curent în poziția pe care acesta ar trebui să o ocupe, față de elementele deja sortate.

Complexitate:

$O(n^2)$ – cazul mediu și cazul defavorabil

$O(n)$ – cazul favorabil

2.3 Merge Sort

Algoritmul Merge Sort a fost introdus de către John von Neumann, în lucrarea sa din anul 1945. [3]

Algoritmul este unul recursiv, bazat pe metoda „divide et impera”: acesta împarte șirul în jumătăți, sortează fiecare jumătate, iar apoi interclasează rezultatele.

Complexitate:

$O(n \log n)$ în toate cazurile

2.4 Quick Sort

Algoritmul Quick Sort a fost inventat în anul 1959 și publicat în anul 1961 de către John Hoare [2]

Algoritmul alege un element pivot, și partiționează șirul în două subșiruri, astfel încât toate elementele de la stânga pivotului sunt mai mici decât acesta, iar toate elementele de la dreapta sa sunt mai mari decât acesta.

Complexitate:

$O(n \log n)$ – cazul mediu

$O(n^2)$ – când pivotul a fost ales prost

2.5 Radix Sort

Algoritmul Radix Sort își are originea în lucrările lui Herman Hollerith legate de prelucrarea datelor pe cartele perforate, în 1889. [4]

Algoritm ce nu se bazează pe comparații între elemente ale tabloului de sortat, ci sortează cifrele numerelor de la cea mai puțin semnificativă la cea mai semnificativă.

Complexitate:

$O(n \cdot k)$ – unde n reprezintă numărul de elemente, iar k numărul de cifre ale celei mai mari valori.

Algoritm	Caz favorabil	Caz mediu	Caz defavorabil
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Radix Sort	$O(n * k)$	$O(n * k)$	$O(n * k)$

Tabela 1: Complexitățile algoritmilor de sortare folosiți

3 Implementare

Algoritmii de sortare prezentați mai sus au fost implementați în limbajul de programare C++. Codul a fost organizat într-un singur fișier (main.cpp), și include:

- Implementările celor cinci algoritmi de sortare ca funcții independente;
- Un sistem de generare a datelor de test (liste nesortate, sortate, plate, aproape sortate);
- Un mecanism de măsurare a timpului de execuție folosind funcții din librăria chrono;
- Meniu interactiv pentru selectarea tipului de test dorit;

Implementarea și datele utilizate pentru experiment pot fi accesate utilizând repozițorul:

https://github.com/florianbudurean/proiect_SA

3.1 Reprezentarea datelor

Datele de test sunt reprezentate ca tablouri de numere întregi, alocate dinamic (int*). Șirul de date de intrare este citit dintr-un fișier text, și conține pe prima linie numărul de elemente, n, urmat de n valori separate prin spații.

3.2 Generarea datelor de test

Există trei tipuri de generare de date de test:

Listă nesortată, unde elementele sunt generate aleator cu funcția `rand()`;

Listă sortată, unde elementele sunt generate crescător având un pas aleator;

Listă plată, unde elementele sunt ordonate crescător, dar cu un pas mic (0 sau 1);

De asemenea, este implementată și opțiunea de a utiliza datele folosite la rularea anterioară a programului.

Pentru experimente, s-au generat liste de dimensiuni: 10, 20, 100, 500, 1.000, 5.000, 10.000, 50.000, 100.000, 500.000, 1.000.000 elemente. Pentru listele mici (10-1.000 de elemente), experimentul a fost repetat de până la 10.000 ori pentru a obține media timpului.

3.3 Măsurarea timpului

Timpul de execuție este măsurat prin funcția `benchmark()`. Aceasta utilizează `chrono::high_resolution_clock` din biblioteca `chrono`. Aceasta oferă cea mai mare precizie disponibilă pe platforma de rulare, de ordinul nanosecundelor, spre deosebire de `clock()` din biblioteca `ctime`, care are o rezoluție limitată la `CLOCKS_PER_SEC` (1.000-1.000.000 ticks). Această metodă permite măsurarea precisă a timpului de execuție a programului.

Funcția `benchmark()` se folosește de pointeri la funcții pentru a putea rula mai mulți algoritmi de sortare. Aceasta primește un pointer la funcția aleasă pentru sortare, tabloul original, dimensiunea n și numărul de rulări `runs`. La fiecare iterație au loc următoarele operații:

1. Se copiază tabloul original într-un tablou buffer temporar, *copyăi*, astfel încât fiecare rulare se face cu aceleași date de intrare.
2. Se înregistrează momentul de start cu `high_resolutionclock::now()`;

3. Se apelează funcția de sortare pe `copy`
4. Se înregistrează momentul de stop și se calculează durata, în secunde, prin diferența dintre timpul de stop și timpul de start.
5. Durata este adăugată la suma totală.

La final, funcția returnează media aritmetică a timpilor ($\text{total} / \text{runs}$), exprimată în secunde. Această medie este apoi furnizată utilizatorului.

3.4 Precizări

De menționat este faptul că algoritmi cu complexitatea $O(n^2)$ sunt dezactivați automat pentru dimensiuni foarte mari (Bubble Sort pentru $n > 100.000$, și Insertion Sort pentru $n > 75.000$). Această caracteristică a fost adoptată pentru a preveni timpii de rulare excesivi, ce blocau calculatorul pe care se rula experimentul.

Pentru liste relativ mici, ($n \leq 1.000$), numărul de rări este setat la 10.000, astfel încât se obține o medie statistică stabilă. Pentru liste mari ($n = 1.000.000$), 10 rări sunt suficiente pentru a furniza un rezultat satisfăcător.

Rezultatele rărilor vor fi prezentate mai jos, în tabele și grafice.

4 Studiu de caz

4.1 Condiții de rulare

Experimentul a fost realizat pe un sistem cu următoarele specificații:

Laptop: Lenovo Ideapad Slim 5i
 Procesor: Intel Core i5-12450H (2.00 GHz)
 Memorie: RAM 16 GB, 4800 MT/s
 Sistem de operare: Windows 11

4.2 Rezultate

Rezultatele experimentului, exprimate în secunde. De menționat faptul că am folosit „–” pentru a marca cazurile în care rularea nu a fost posibilă, datorită complexității prea mari a algoritmului în cazul respectiv.

4.2.1 Liste cu elemente aleatoare

n	Rări	Bubble Sort	Insertion Sort	Merge Sort	Quick Sort	Radix Sort
10	1,000	0.00000083090	0.00000041180	0.00000243050	0.00000056900	0.00000228980
100	1,000	0.00004215130	0.00000880020	0.00002791690	0.00000654560	0.00000937210
500	1,000	0.00105028000	0.00017530200	0.00012894900	0.00003892330	0.00005070820
1,000	1,000	0.00387247000	0.00067395800	0.00031699100	0.00013400400	0.00008413110
10,000	100	–	0.06341190000	0.00374591000	0.00192922000	0.00072177500
100,000	10	–	–	0.04796050000	0.02504180000	0.00759991000
500,000	10	–	–	0.24794300000	0.12790700000	0.04424070000
1,000,000	10	–	–	0.48232400000	0.26321600000	0.08591120000

Tabela 2: Rezultatele testelor pe date nesortate

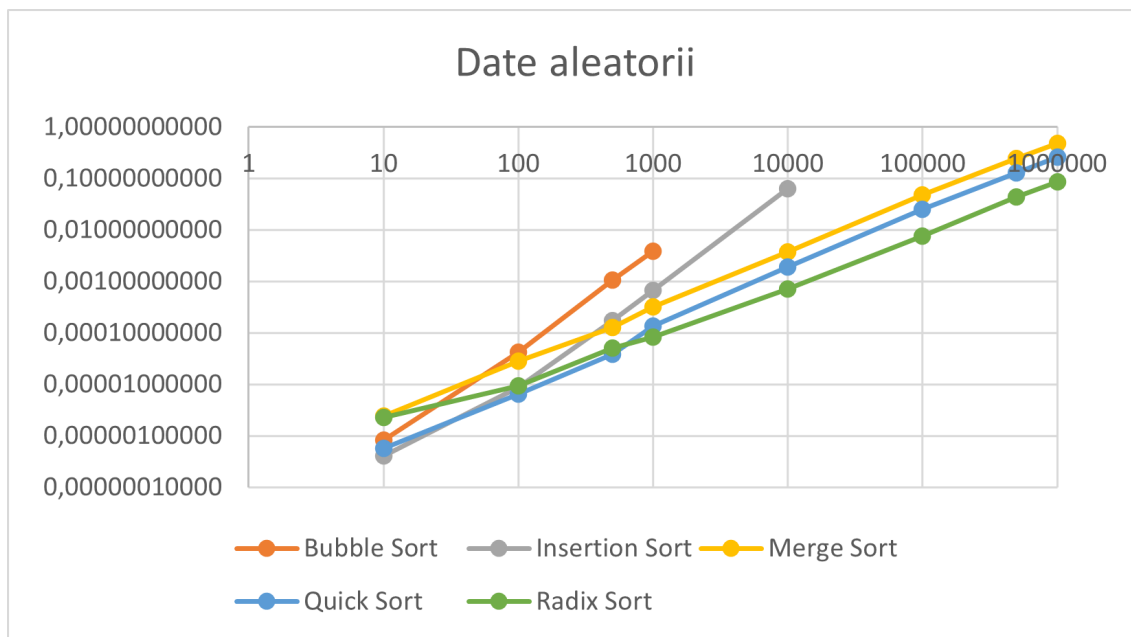


Figura 1: Reprezentare grafică a timpilor de execuție pentru date nesortate

4.2.2 Liste sortate crescător

n	Rulări	Bubble Sort	Insertion Sort	Merge Sort	Quick Sort	Radix Sort
10	1,000	0.0000006580	0.0000003367	0.0000022375	0.0000010874	0.0000011795
100	1,000	0.0000141576	0.0000006877	0.0000230461	0.0000415009	0.0000068016
500	1,000	0.0002474810	0.0000019914	0.0001137060	0.0008805910	0.0000350818
1,000	1,000	0.0010839200	0.0000043547	0.0002517260	0.0034675400	0.0000648021
10,000	100	–	0.0000334340	0.0026191100	0.3979250000	0.0014662100
100,000	10	–	–	0.0284902000	–	0.0095122500
500,000	10	–	–	0.1421920000	–	0.0692472000
1,000,000	10	–	–	0.2883410000	–	0.1243750000

Tabela 3: Rezultatele testelor pe date sortate

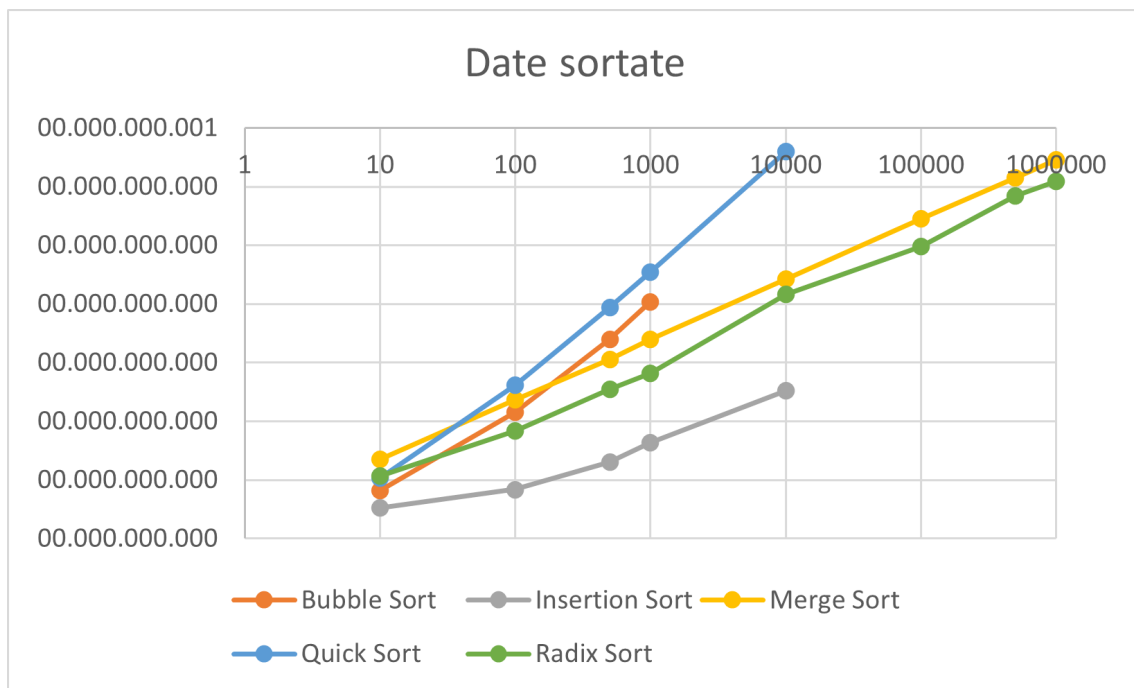


Figura 2: Reprezentare grafică a timpilor de execuție pentru date sortate

4.2.3 Liste plate

n	Rulări	Bubble Sort	Insertion Sort	Merge Sort	Quick Sort	Radix Sort
10	1,000	0.00000048250	0.00000034980	0.00000207490	0.00000077490	0.00000136490
100	1,000	0.00001341860	0.00000066510	0.00002157900	0.00002270820	0.00000463160
500	1,000	0.00025638500	0.00000225980	0.00012359200	0.00050227600	0.00002753040
1,000	1,000	0.00105305000	0.00000386790	0.00022456500	0.00198348000	0.00005280890
10,000	100	–	0.00003401300	0.00262287000	0.23440900000	0.00117755000
100,000	10	–	–	0.02732520000	–	0.00930172000
500,000	10	–	–	0.14429900000	–	0.06622430000
1,000,000	10	–	–	0.29382300000	–	0.11748600000

Tabela 4: Rezultatele testelor pe date plate

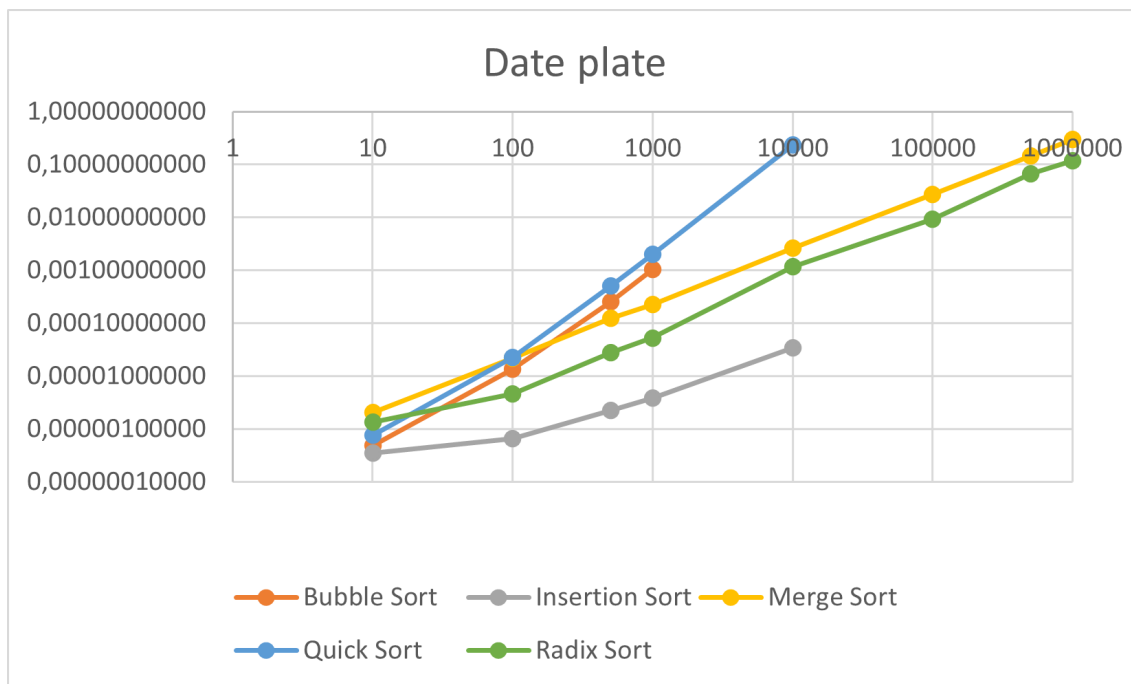


Figura 3: Reprezentare grafică a timpilor de execuție pentru date plate

4.3 Analiza rezultatelor și concluzii

4.3.1 Liste aleatoare

Pentru datele sortate aleator, se observă diferențele cele două categorii de algoritmi: Algoritmi de complexitate $O(n^2)$: Bubble Sort, Insertion Sort

Algoritmi eficienți: Merge Sort, Quick Sort, Radix Sort

Pentru dimensiuni reduse, sub 100 de elemente, diferențele sunt reduse, iar uneori algoritmi simpli sunt chiar competitivi.

Însă, pe măsură ce n crește, se observă diferențe notabile în performanța algoritmilor, astfel:

Bubble Sort și Insertion Sort devin rapid ineficiente, pentru valori $n = 10.000$ (Insertion Sort 0,063 s)

Merge Sort și Quick Sort rămân eficiente (0,001-0,004 s) Radix sort devine cel mai rapid pentru dimensiuni mari. Spre exemplu, pentru $n = 1.000.000$, timpul de execuție este doar 0,086 s, mai rapid decât Quick Sort cu 0,177 s.

4.3.2 Liste sortate crescător

Acesta este cazul favorabil pentru algoritmii de complexitate $O(n^2)$, Bubble și Insertion sort, deoarece algoritmi doar parcurg odată șirul și verifică proprietatea de sortare, fără a efectua modificări. În cadrul implementării curente, pentru $n = 1000$, Bubble Sort este de aproximativ 3 ori mai eficient decât Quick Sort (0,001 secunde vs. 0,003 secunde)

Algoritmi ultra eficienți, precum Quick Sort, dau rezultate contraintuitive așteptărilor. Spre exemplu, pentru $n = 10.000$, rezultă un timp de execuție de 0,38 secunde, iar pentru dimensiuni mai mari de 50.000, algoritmul Quick Sort eșuează.

Trebuie menționat faptul că Merge Sort și Radix Sort rămân stabile și nu sunt influențate de ordinea în care se află elementele de sortat.

De asemenea, trebuie menționat faptul că comportament anormal al algoritmului Quick Sort este cauzat de metoda de alegere a pivotului, în cazul nostru chiar ultimul element din vectorul de sortat. Desigur că am fi putut alege o abordare diferită pentru alegerea pivotului, precum elementul din mijloc sau media a 3 elemente aleatoare din vector. O simplă verificare inițială a

stării de sortare a tabloului ar fi putut evita această situație. Am ales însă această abordare pentru a sublinia anomaliile ce apar în aceste cazuri particulare.

4.3.3 Liste Plate

În cazul rulării algoritmilor cu date de intrare reprezentate de liste plate, observăm un comportament asemănător cu cel ce se poate regăsi la listele sortate crescător, datorită faptului că multe elemente sunt egale sau apropiate unele de celelalte.

4.4 Observații

Observăm faptul că, așa cum este așteptat, timpul de execuție pentru Bubble Sort și Insertion Sort cresc rapid, acesta fiind un comportament tipic algoritmilor de complexitate pătratică. De asemenea, Merge Sort și Quick Sort au complexitatea $n * \log(n)$, iar Radix Sort are o performanță aproape liniară.

Notabil este și faptul că, pentru $n = 10$, Merge Sort și Radix Sort sunt mai lente decât ceilalți algoritmi, motivul fiind costuri ce nu pot fi evitate (alocări, recursivitate).

Quick Sort variază puternic în funcție de datele de input și tipul sortării acestora, în timp ce Merge Sort și Radix Sort au o performanță constantă.

4.5 Limitările experimentului

Rezultatele obținute depind de mediul hardware și software utilizat, precum și de detaliile particulare ale implementării. De exemplu, în cazul algoritmului Quick Sort, alegerea ultimului element drept pivot (în mod intenționat, pentru a sublinia limitările sale) influențează negativ performanța pe liste deja sortate sau aproape sortate.

De asemenea, experimentul analizează în principal timpul de execuție, fără a măsura în mod direct memoria suplimentară utilizată. Acest aspect este relevant mai ales pentru Merge Sort și Radix Sort, care folosesc structuri auxiliare.

O altă limitare este reprezentată de faptul că, în cazul nostru, datele de intrare sunt formate numai din numere întregi. Astfel, rezultatele pot fi diferite în cazul altor tipuri de date sau în cazul unor implementări optimizate din biblioteci standard.

5 Comparație cu literatura

Rezultatele experimentale obținute sunt, în mare parte, în concordanță cu literatura de specialitate. Lucrări clasice precum cele ale lui Knuth [1] și Cormen et al. [5] prezintă complexitățile teoretice ale algoritmilor de sortare și evidențiază diferențele dintre metodele cu complexitate pătratică și cele cu complexitate $O(n \log n)$ sau aproape liniară.

Studii comparative mai recente tratează algoritmii de sortare dintr-o perspectivă practică, apropiată de cea folosită în această lucrare. Al-Kharabsheh et al. [6] compară algoritmi clasici precum Selection Sort, Quick Sort, Insertion Sort, Merge Sort și Bubble Sort, urmărind criterii precum timpul de execuție, stabilitatea și spațiul de memorie utilizat. Studiul evidențiază faptul că evaluarea unui algoritm de sortare nu trebuie făcută doar pe baza complexității teoretice, ci și prin raportare la performanța practică pe diferite seturi de date.

Hammad [7] realizează o comparație experimentală între mai mulți algoritmi de sortare, implementați în limbajul C#. Scopul principal al studiului este măsurarea timpului de execuție în funcție de numărul de elemente de intrare. Rezultatele prezentate arată că nu există un algoritm optim în mod absolut pentru toate situațiile, performanța depinzând de dimensiunea datelor și de condițiile concrete de rulare.

Gill et al. [8] compară mai mulți algoritmi de sortare, printre care Bubble Sort, Insertion Sort, Selection Sort, Counting Sort, Radix Sort și Bucket Sort. Studiul face diferența între algoritmi bazați pe comparații și algoritmi care nu se bazează pe comparații directe între elemente. Autorii subliniază că alegerea algoritmului potrivit depinde de intervalul valorilor, distribuția datelor și tipul problemei de rezolvat.

În cazul algoritmilor Bubble Sort și Insertion Sort, rezultatele confirmă comportamentul descris în literatură. Acești algoritmi sunt simpli și pot fi eficienți pentru dimensiuni mici sau pentru date deja sortate, însă devin rapid ineficienți pe măsură ce numărul de elemente crește.

Pentru Merge Sort, rezultatele obținute confirmă stabilitatea algoritmului. Timpii de execuție cresc într-un mod predictibil, iar performanța nu este afectată semnificativ de ordinea inițială a elementelor. Acest comportament corespunde complexității teoretice $O(n \log n)$ în toate cazurile.

În cazul Quick Sort, rezultatele arată diferența dintre analiza teoretică medie și comportamentul practic al unei implementări concrete. Pe date aleatoare, algoritmul este eficient, însă pe liste sortate sau plate performanța scade semnificativ din cauza alegerii ultimului element drept pivot. Acest rezultat confirmă faptul că implementarea și distribuția datelor de intrare pot influența puternic performanța reală.

Radix Sort a obținut cei mai buni timpi pentru volume mari de date numerice. Acest rezultat este în concordanță cu literatura, care arată că algoritmii care nu se bazează pe comparații pot depăși limita $O(n \log n)$ în anumite condiții, mai ales atunci când valorile au un număr limitat de cifre.

Prin comparație cu studiile existente, contribuția acestei lucrări nu constă în propunerea unui algoritm nou, ci în realizarea unei analize experimentale proprii, într-un mediu concret de rulare, folosind aceleași condiții pentru toți algoritmii testați.

Din perspectiva literaturii, problema sortării este deja rezolvată la nivel teoretic. Sunt cunoscute complexitățile algoritmilor clasici, astfel:

- Bubble Sort și Insertion Sort au complexitate medie și defavorabilă de ordinul $O(n^2)$
- Merge Sort are complexitate $O(n \log n)$ în toate cazurile
- Quick Sort are complexitate medie $O(n \log n)$, dar poate ajunge și la complexitate de $O(n^2)$ în cazul în care pivotul nu a fost ales corect.
- Radix Sort are un comportament aproape liniar pentru date numerice cu un număr limitat de cifre.

5.1 Problemă

Voi trata aceeași problemă fundamentală ca literatura: ordonarea cât mai eficientă a unui set de date. Voi analiza comportamentul practic al algoritmilor aleși, și nu voi căuta să demonstrez corectitudinea algoritmilor.

5.2 Context

Literatura analizează algoritmii d.p.d.v. abstract, independent de limbaj de programare, program de compilare sau hardware utilizat. Eu voi analiza însă algoritmii într-un mediu practic, descris în cele ce urmează.

5.3 Metodă

În timp ce literatura este caracterizată de teoria complexității și analiza teoretică, eu mă voi axa pe experiment. Acesta presupune generarea mai multor tipuri de date de intrare, rularea algoritmilor pe aceleași seturi de date și calcularea timpilor medii de execuție. În lucrarea sa, Hammad [7] implementează un număr de algoritmi, mai mare decât în acest caz, în C#. Acesta, de asemenea, măsoară timpul de execuție al algoritmilor.

6 Concluzii

Lucrarea a avut ca scop analiza experimentală a performanței unor algoritmi clasici de sortare: Bubble Sort, Insertion Sort, Merge Sort, Quick Sort și Radix Sort. Pentru acest lucru, algoritmii au fost implementați în C++, au fost generate mai multe tipuri de date de intrare, iar timpii de execuție au fost măsurați folosind biblioteca `chrono`.

Rezultatele obținute confirmă în mare parte ceea ce este cunoscut din teoria algoritmilor. Bubble Sort și Insertion Sort au rezultate bune doar pentru dimensiuni mici sau pentru cazuri favorabile, dar devin rapid ineficienți atunci când numărul de elemente crește. Merge Sort are un comportament stabil, indiferent de ordinea inițială a datelor. Quick Sort este eficient pe date aleatoare, dar poate avea rezultate slabe pe date deja sortate sau aproape sortate, dacă pivotul este ales nefavorabil. Radix Sort a avut cei mai buni timpi pentru volume mari de date numerice.

În timpul realizării aplicației au apărut și câteva dificultăți. Una dintre ele a fost timpul foarte mare de execuție al algoritmilor cu complexitate $O(n^2)$ pentru valori mari ale lui n . Această problemă a fost adresată prin dezactivarea automată a rulării Bubble Sort și Insertion Sort pentru dimensiuni foarte mari, astfel încât programul să nu blocheze sistemul.

O altă dificultate a fost măsurarea timpilor de execuție pentru liste mici, deoarece diferențele dintre algoritmi sunt foarte mici. Pentru a obține rezultate mai stabile, testele au fost repetate de mai multe ori, iar timpul final a fost calculat ca medie aritmetică a rulărilor.

O problemă observată în timpul testelor a fost comportamentul nefavorabil al Quick Sort pe liste sortate sau plate. Acest lucru a apărut din cauza metodei de alegere a pivotului, în cazul nostru ultimul element din vector. Problema nu a fost eliminată complet, dar a fost analizată și folosită pentru a arăta cât de mult poate influența o alegere de implementare performanța reală a unui algoritm.

Problemele care nu au fost complet depășite pot fi considerate deschise. O posibilă îmbunătățire ar fi modificarea algoritmului Quick Sort prin alegerea aleatoare a pivotului sau prin metoda median-of-three. De asemenea, experimentul ar putea fi extins prin măsurarea memoriei folosite de fiecare algoritm, nu doar a timpului de execuție.

În viitor, analiza ar putea fi completată prin testarea unor seturi de date mai variate, prin compararea implementărilor proprii cu funcțiile optimizate din biblioteca standard C++ și prin rularea testelor pe sisteme hardware diferite. S-ar putea obține astfel o imagine mai completă asupra performanței algoritmilor de sortare în situații practice diferite.

Bibliografie

- [1] Donald E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, Addison-Wesley, 1973.
- [2] C. A. R. Hoare, "Quicksort", *The Computer Journal*, vol. 5, no. 1, pp. 10–16, 1962.
- [3] John von Neumann, "First Draft of a Report on the EDVAC", 1945.
- [4] Herman Hollerith, "An Electric Tabulating System", Columbia University, 1889.
- [5] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms, Third Edition*, MIT Press, 2009.
- [6] Khalid Suleiman Al-Kharabsheh, Ibrahim Mahmoud AlTurani, Abdallah Mahmoud Ibrahim AlTurani, Nabeel Imhammed Zanoon, *Review on Sorting Algorithms: A Comparative Study*, International Journal of Computer Science and Security, Volume 7, Issue 3, 2013.
- [7] Jehad Hammad, *A Comparative Study between Various Sorting Algorithms*, IJCSNS International Journal of Computer Science and Network Security, Volume 15, Number 3, 2015.
- [8] Sandeep Kaur Gill, Virendra Pal Singh, Pankaj Sharma, Durgesh Kumar, *A Comparative Study of Various Sorting Algorithms*, International Journal of Advanced Studies of Scientific Research, Volume 4, Number 1, 2019.